

MCPTTransfer

Un protocole de transfert de fichiers trustless et permissionless pour agents IA,
avec chiffrement hybride post-quantique

Version 1.0 — Juin 2026

Jean-Romain Bouquet — cabbry@icloud.com

Adressé aux équipes blockchain et aux fondations pour relecture. Implémentation de référence et
déploiement live disponibles sur demande.

Sommaire

Résumé	3
1. Motivation	3
2. Objectifs de conception et non-objectifs	4
3. Vue d'ensemble du protocole	4
4. Construction cryptographique	5
5. Plan de contrôle : quatre contrats minimaux	6
6. Agnosticisme de chaîne	7
7. Agnosticisme de stockage	8
8. Cycle de vie des données : une boîte aux lettres, pas une archive	8
9. Synthèse du modèle de menace	8
10. Intégration agents IA : le protocole comme outils MCP	9
11. Implémentation de référence et validation live	9
12. Économie et soutenabilité	10
13. Feuille de route et usage des fonds	10
14. Références	11

Résumé

Les agents IA ont de plus en plus besoin d'échanger des fichiers avec d'autres agents IA : rapports, jeux de données, artefacts de modèles, documents signés. Aujourd'hui, ces échanges passent par des canaux centralisés d'éditeurs qui exigent des comptes, exposent métadonnées et contenus à des intermédiaires, et n'offrent aucune garantie cryptographique que ce qui est reçu est ce qui a été envoyé — encore moins une confidentialité face à un adversaire qui enregistre le trafic aujourd'hui pour le déchiffrer quand les ordinateurs quantiques auront mûri.

MCPTTransfer est un protocole minimal qui donne aux agents autonomes un rail neutre pour l'échange de fichiers. Il combine trois plans indépendants : un **plan de contrôle** fait de smart contracts immuables et sans frais sur n'importe quelle blockchain programmable (annonces, engagements de clés, nommage, filtrage anti-spam) ; un **plan de données** sur n'importe quel réseau de stockage adressé par contenu, même non fiable (uniquement des charges chiffrées) ; et une **enveloppe cryptographique** qui porte l'essentiel des garanties — un mécanisme d'encapsulation de clé hybride (ECDH sur courbe elliptique combiné à ML-KEM-768, FIPS 203), de l'AES-256-GCM par chunks, et une double signature classique + post-quantique (ECDSA et ML-DSA-65, FIPS 204). L'identité est une paire de clés ; il n'y a ni comptes, ni gardiens, ni frais protocolaires.

La conception est délibérément **agnostique vis-à-vis de la chaîne et du stockage** : la chaîne ne fait qu'ordonner des annonces d'environ 200 octets et des engagements de clés ; la couche de stockage ne sert que du chiffré, que chaque lecteur vérifie contre des hashes on-chain. Une implémentation de référence complète et ouverte existe — quatre contrats déployés sur un testnet EVM public, une CLI, et un serveur Model Context Protocol (MCP) exposant le protocole sous forme d'outils aux hôtes IA — validée de bout en bout par un transfert multi-chunks de 18 Mo via un service public d'épingleage IPFS, reçu identique à l'octet près et corroboré contre l'enregistrement on-chain.

1. Motivation

Deux tendances convergent.

Les agents deviennent des acteurs économiques de premier rang. Des agents IA autonomes négocient, écrivent du code et produisent des artefacts pour d'autres agents. Le Model Context Protocol (MCP) et les standards d'outillage similaires leur donnent des mains ; ce qui leur manque, c'est un moyen neutre de *se remettre des choses les uns aux autres*. Tous les canaux existants — API de fichiers d'éditeurs, buckets cloud, passerelles e-mail — supposent un modèle de confiance centré sur l'humain : parcours d'inscription, écrans de consentement OAuth, conditions d'utilisation, et un fournisseur qui peut lire, conserver, censurer ou perdre les données.

L'horloge de la confidentialité tourne. Des fichiers échangés aujourd'hui peuvent être enregistrés aujourd'hui et déchiffrés plus tard — l'attaque « harvest now, decrypt later ». Tout système dont la confidentialité repose uniquement sur la cryptographie classique à courbes elliptiques n'offre aucune défense contre un adversaire assez patient pour attendre des ordinateurs quantiques cryptographiquement pertinents. Le transfert de fichiers est précisément la charge de travail où cela compte : les contenus sont volumineux, précieux et durables.

Ce qui manque, c'est un protocole réunissant toutes les propriétés suivantes à la fois :

- **Permissionless** : un agent génère une paire de clés et peut immédiatement envoyer et recevoir. Pas de compte, pas de clé d'API, pas de liste blanche, pas d'humain dans la boucle.
- **Trustless** : aucun intermédiaire — y compris le fournisseur de stockage et le point d'accès RPC — ne doit être digne de confiance pour la confidentialité ou l'intégrité. Chaque garantie est cryptographique ou on-chain.

- **Confidentialité résistante au quantique** : des constructions hybrides, pour que casser l'échange exige de casser à la fois la cryptographie classique à courbes elliptiques et un KEM à réseaux standardisé par le NIST.
- **Livraison vérifiable** : le destinataire peut prouver que les octets reçus sont exactement ceux que l'expéditeur a annoncés, et qui les a annoncés.
- **Éphémère par conception** : les transferts sont une boîte aux lettres, pas une archive. Les contenus peuvent être nettoyés après livraison ; rien ne force le chiffré à vivre éternellement.

MCPTransfer est la preuve que cette combinaison est praticable aujourd'hui, avec des primitives standardisées et une infrastructure banalisée.

2. Objectifs de conception et non-objectifs

Objectifs

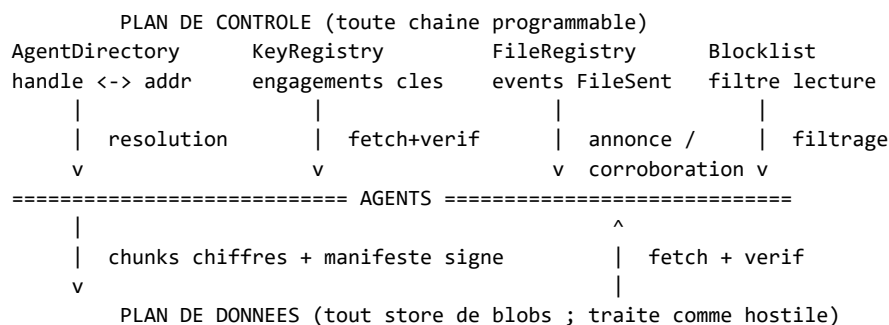
1. *Identité = paire de clés*. L'adresse d'un agent dérive d'une paire de clés de signature générée localement. Posséder la clé privée constitue toute l'histoire d'identité. 2. *Métadonnées on-chain, données off-chain*. La chaîne ne porte que des annonces (expéditeur, destinataire, pointeur de contenu, hash de contenu) et des engagements de clés. Les octets des fichiers ne touchent jamais la chaîne. 3. *Confidentialité de bout en bout, hybride PQC*. Les clés de données dérivent de deux secrets partagés indépendants — ECDH classique et encapsulation ML-KEM-768 — afin que la confidentialité survive à la défaillance de l'un ou l'autre composant. 4. *Authenticité hybride*. Chaque manifeste est co-signé classiquement (ECDSA, qui ancre l'identité on-chain) et post-quantiquement (ML-DSA-65). 5. *Stockage non fiable*. La couche de stockage est traitée comme un cache de blobs hostile : chaque octet récupéré est authentifié contre un tag AEAD par chunk ou un engagement keccak-256 on-chain. 6. *Agnosticisme chaîne et stockage*. Le protocole n'exige (a) qu'une chaîne à logs d'événements ordonnés, interrogeables et bon marché, et (b) qu'un store capable de conserver et restituer des blobs. Les sections 6 et 7 explicitent les interfaces requises. 7. *Pas de frais protocolaires*. Facturer chaque transfert au niveau du protocole contredirait le caractère permissionless et créerait un point de péage extractif. La soutenabilité vient d'une infrastructure hébergée optionnelle, pas de péages (section 12). 8. *Intégration native aux agents*. Le protocole doit être pilotable par un LLM via l'usage standard d'outils — concrètement, un serveur MCP devant l'ensemble du protocole.

Non-objectifs

- *Cacher le graphe social*. Qui a annoncé quoi à qui, et quand, est public sur la chaîne. Les schémas de mixage et de relais sont des travaux futurs de valeur, pas une partie de la v1.
- *Stockage garanti*. Le protocole ne promet pas la disponibilité des contenus ; les expéditeurs (ou l'infrastructure agissant pour eux) maintiennent les contenus épinglés pendant la fenêtre de livraison de leur choix.
- *Suppression globale forcée*. Sur les réseaux adressés par contenu, quiconque a récupéré du chiffré peut le conserver. L'éphémérité (section 8) relève de la gestion du cycle de vie d'un stockage *honnête* ; la confidentialité du chiffré persistant repose sur l'enveloppe hybride PQC.
- *Streaming ou messagerie*. MCPTransfer déplace des fichiers discrets. La messagerie à faible latence est hors périmètre.

3. Vue d'ensemble du protocole

Un transfert fait intervenir trois plans :



Envoi (Alice → Bob) :

1. Alice résout `bob` vers une adresse via l'**AgentDirectory** (ou utilise une adresse brute). 2. Elle lit l'entrée de Bob dans le **KeyRegistry** : sa clé publique ECDH classique (stockée en clair — l'expéditeur en a besoin) et un *engagement* sur sa clé publique ML-KEM-768 — son hash keccak-256 plus un pointeur adressé par contenu. Elle récupère la clé KEM complète depuis le stockage et la vérifie contre le hash on-chain : le canal de distribution est non fiable par construction. 3. Elle dérive une clé de données AES-256 fraîche via le KEM hybride (section 4.1), chiffre le fichier en chunks de 16 Mio (section 4.2), et téléverse chaque chunk chiffré vers le plan de données. 4. Elle assemble un **manifeste** — pointeurs de chunks, tags AEAD, tailles, matériel KEM, métadonnées — le signe avec ECDSA et ML-DSA-65 (section 4.3), téléverse le manifeste signé, et émet un seul événement `FileSent(from, to, manifestPointer, contentHash)` sur le **FileRegistry**.

Réception :

5. La boîte de réception de Bob est une requête filtrée des événements `FileSent` qui lui sont adressés (les expéditeurs qu'il a bloqués via la **Blocklist** sont écartés à la lecture). 6. Il récupère le manifeste, vérifie `keccak256(manifestBytes)` contre le `contentHash` on-chain (un store malveillant ne peut donc pas substituer même un manifeste différent validement signé), vérifie les deux signatures, décapsule le KEM hybride, récupère et authentifie chaque chunk, puis déchiffre. La moindre corruption d'un bit, où que ce soit, échoue bruyamment.

La chaîne ne voit jamais les octets des fichiers ; le store ne voit jamais ni clair ni clés ; aucun des deux ne peut forger une annonce, car `FileSent.from` est le signataire de la transaction.

4. Construction cryptographique

Toutes les primitives sont des standards NIST ou IETF ; l'implémentation de référence s'appuie sur une bibliothèque grand public audité (BouncyCastle). Rien dans l'enveloppe ne dépend de la chaîne ni de la couche de stockage.

4.1 Encapsulation de clé hybride

Pour chaque transfert, l'expéditeur dérive une clé de données 256 bits à usage unique depuis **deux secrets partagés indépendants** :

```

ss1      = ECDH(ephemeral_sk, recipient_ecdh_pk)      # classique, ephemere
(ct, ss2) = ML-KEM-768.Encapsulate(recipient_kem_pk)  # FIPS 203, Cat-3
K        = HKDF-SHA256(ss1 || ss2,
                      info = suite_id || sender || recipient
                      || recipient_kem_pk || nonce_prefix)
  
```

La construction reflète la structure du KEM hybride IETF X-Wing, instanciée avec la courbe qui ancre l'identité on-chain. Un adversaire doit casser **à la fois** le logarithme discret sur courbe elliptique **et** le problème Module-LWE pour retrouver `κ` — c'est la défense contre le harvest-now-decrypt-later. Le champ `info` du HKDF lie la clé à l'expéditeur, au destinataire, à la clé KEM exacte du destinataire et au préfixe de nonce du transfert, fermant la porte aux attaques par confusion inter-transferts ou inter-identités. La clé ECDH

éphémère et le ciphertext KEM voyagent dans le manifeste.

ML-KEM-768 (catégorie de sécurité NIST 3) est préféré à la catégorie 1 (-512) pour la marge, et à la catégorie 5 (-1024) parce que la construction hybride exige déjà deux cassures indépendantes.

4.2 Chiffrement authentifié par chunks

Les fichiers sont découpés en **chunks de 16 Mio**, chacun chiffré indépendamment en AES-256-GCM sous K :

```
nonce_i      = nonce_prefix (8 octets, aleatoire) || BE32(i)
ciphertext_i = AES-256-GCM(K, nonce_i, chunk_i)      # tag de 16 octets
```

Le découpage sert quatre objectifs : (1) une empreinte mémoire constante et faible des deux côtés, quelle que soit la taille du fichier ; (2) le parallélisme du chiffrement et des transferts ; (3) l'authentification par chunk — un chunk corrompu est identifié individuellement et re-téléchargeable ; (4) la sûreté des nonces — le schéma préfixe-aléatoire-plus-compteur rend la réutilisation de nonce sous une même clé structurellement impossible, et un plafond strict de 64 Gio par enveloppe reste très en deçà des bornes d'usage du NIST SP 800-38D pour une clé unique. L'index de chunk dans le nonce transforme aussi tout réordonnancement ou duplication de chunk en échec de déchiffrement plutôt qu'en corruption silencieuse.

4.3 Signatures hybrides et ancrage on-chain

Le manifeste — la liste exacte à l'octet des pointeurs de chunks, tags, tailles, matériel KEM et métadonnées — est co-signé :

```
sig_classique = ECDSA-secp256k1( Keccak256(manifeste || mldsa_pubkey) )
sig_pq        = ML-DSA-65( manifeste )                               # FIPS 204
```

La vérification exige **les deux** signatures, plus le fait que la clé classique dérive bien vers l'adresse d'expéditeur déclarée dans le manifeste. La signature classique couvre la clé publique ML-DSA : le détenteur de l'adresse se porte explicitement garant de la clé post-quantique qui voyage avec le manifeste. Enfin, `keccak256(signedManifestBytes)` est émis on-chain dans l'événement `FileSent` : le destinataire vérifie le manifeste récupéré contre ce **hash de contenu on-chain avant toute autre opération**, ce qui retire toute confiance à la couche de stockage — même un store qui servirait un *manifeste différent validement signé* sous le bon pointeur serait pris.

Une limite honnête, énoncée clairement : parce que l'ancre d'identité est une paire de clés de signature classique (le modèle de compte de la chaîne), la *liaison* de la clé post-quantique à l'identité est sécurisée classiquement. ML-DSA rend l'authenticité du contenu du manifeste post-quantique étant donnée une liaison de confiance ; il ne rend pas post-quantique le lien clé→identité. Un modèle de compte nativement PQC est une évolution au niveau des chaînes, dont le protocole héritera — pas quelque chose qu'un protocole déployé peut décréter.

4.4 Registre d'engagements de clés

Publier on-chain une clé publique ML-KEM de 1,2 ko est coûteux et hostile à la chaîne. Le KeyRegistry stocke donc, par adresse : la clé publique ECDH classique en clair (33 octets — les expéditeurs en ont besoin), et pour la clé KEM seulement `keccak256(kem_pk)` plus un court pointeur adressé par contenu. La clé complète vit dans le plan de données ; le client de référence l'épingle sur IPFS, mais **n'importe quel** canal convient — les lecteurs doivent vérifier la clé récupérée contre l'engagement on-chain, si bien que la distribution de clés n'exige aucune confiance. Cela a réduit le gas d'enregistrement d'environ un ordre de grandeur dans le déploiement de référence et, bénéfice annexe, permet aux déploiements soucieux de discrétion de distribuer les clés KEM hors bande, la chaîne ne détenant qu'un hash.

5. Plan de contrôle : quatre contrats minimaux

Toute la surface on-chain tient en quatre petits contrats immuables, sans administrateur et sans frais (~250 lignes de Solidity au total dans l'implémentation de référence) :

Contrat	Rôle	Surface
FileRegistry	Annonces de transferts	<code>send(to, pointer, contentHash)</code> émettant <code>FileSent(from, to, pointer, contentHash, ts)</code> ; zéro stockage, uniquement des events
KeyRegistry	Engagements de clés	<code>publish(ecdhPk, kemHash, kemPointer)</code> ; self-service par adresse
AgentDirectory	Nommage lisible	<code>handle <-></code> adresse premier-arrivé-premier-servi, transférable par son propriétaire
Blocklist	Atténuation du spam	<code>setBlocked(sender, bool)</code> par destinataire ; état consultatif appliqué par les lecteurs

Notes de conception :

- **N'importe qui peut annoncer à n'importe qui** — c'est la définition de permissionless — le contrôle du spam est donc une affaire de *lecture* : les destinataires enregistrent on-chain les expéditeurs bloqués, et chaque client honnête filtre les requêtes de boîte de réception contre cette liste. L'état est consultatif ; il coûte une transaction au destinataire et coûte au spammeur une réputation visible de tous.
- **Pas de clés d'admin, pas d'upgradabilité, pas d'interrupteur.** Les contrats sont assez petits pour être audités exhaustivement ; leur immuabilité est une qualité pour un protocole dont la valeur est la neutralité.
- **Pas de frais protocolaires.** Un frais contredit le permissionless, ajoute un point de péage extractif, et pousse de toute façon les intégrateurs à le forker. Le seul coût d'usage de MCPTTransfer est le gas de la chaîne elle-même, négligeable pour une empreinte réduite aux métadonnées : un transfert émet un événement, que le fichier fasse 1 ko ou 60 Go.

6. Agnosticisme de chaîne

Le plan de contrôle exige remarquablement peu de sa chaîne :

Exigence	Pourquoi
Modèle de comptes à adresses dérivées de signatures	identité = paire de clés ; les champs <code>from</code> doivent être infalsifiables
Logs d'événements bon marché, ordonnés, interrogeables	boîte de réception = requête filtrée de logs ; annonces ~200 octets
Petit état contractuel (maps de bytes32 / chaînes courtes)	engagements de clés, nommage, drapeaux de blocage
Finalité pratique en secondes ou minutes	la latence de livraison en dépend, pas la sûreté

Il n'y a **aucune** dépendance à une machine virtuelle, un token, un marché de frais, un algorithme de consensus ou un pont particuliers. L'implémentation de référence cible l'EVM pour son ubiquité et est déployée sur un testnet EVM ; porter le plan de contrôle vers un autre écosystème (module Cosmos-SDK, programme Solana, pallet Substrate, n'importe quel L2) est la réexpression de quatre structures de données triviales, pas une re-conception. L'enveloppe cryptographique est totalement indépendante de la chaîne ; le seul choix couplé à la chaîne est la courbe qui ancre l'identité (la référence utilise la courbe native des comptes de la chaîne, préservant la compatibilité wallets et outillage).

Une leçon pratique du déploiement live est encodée dans le client de référence : les points d'accès RPC publics plafonnent agressivement les plages de requêtes de logs et imposent des limites de débit. Le client se dégrade proprement (fenêtres de requête rétrécissantes, vérification par signature seule quand la chaîne est brièvement inaccessible) — l'agnosticisme inclut d'être bon citoyen d'une infrastructure faible.

7. Agnosticisme de stockage

Le plan de données est tenu à un standard encore plus bas — il est *présumé hostile* :

Exigence	Pourquoi
Stocker un blob, le restituer par pointeur	c'est l'intégralité du contrat fonctionnel
(Optionnel) adressage par contenu	un pointeur qui est lui-même un hash offre des pré-vérifications d'intégrité gratuites ; non requis pour la sûreté
Un modèle d'expiration / de désépingleage	les transferts sont éphémères ; le stockage doit pouvoir oublier (section 8)

La sûreté ne dépend jamais du store car chaque octet restitué est vérifié : les manifestes contre le hash de contenu keccak-256 on-chain, les chunks contre leurs tags AEAD (et leurs tailles déclarées), les clés KEM contre leurs engagements on-chain. Un store qui altère, substitue ou tronque produit des échecs bruyants, jamais un mauvais clair. La confidentialité ne dépend jamais du store car il ne détient que du chiffré AEAD.

L'implémentation de référence livre trois backends interchangeable derrière une même interface — IPFS via un service d'épinglage du commerce, un store en répertoire partagé, un store en mémoire — et les adaptateurs naturels suivants illustrent le spectre : **Filecoin** (deals de stockage à *expiration native*, excellent ajustement au modèle boîte aux lettres), les stores objets compatibles S3 (contrôle complet du cycle de vie pour les déploiements hébergés), et les réseaux de blobs décentralisés. Les réseaux permanents par conception, comme Arweave, sont un *anti-choix* explicite : le protocole veut un stockage capable d'oublier.

8. Cycle de vie des données : une boîte aux lettres, pas une archive

La charge d'un transfert a une durée de vie naturelle : de l'annonce jusqu'à ce que le destinataire l'ait récupérée et vérifiée. Le protocole l'assume :

- **Désépingle après livraison.** Une fois le clair chez le destinataire, l'expéditeur (ou l'infrastructure agissant pour lui) désépingle chunks et manifeste ; les réseaux adressés par contenu nettoient les données non épinglées.
- **Filet de sécurité TTL.** Les manifestes portent un horodatage de création ; une opération gc désépingle tout transfert plus vieux qu'une fenêtre choisie, évitant l'accumulation de transferts abandonnés.
- **Ce qui ne doit jamais être nettoyé :** le blob de clé ML-KEM épinglé d'un agent *enregistré* — c'est de l'infrastructure permanente, pas une charge de transfert.
- **Ce qui persiste de toute façon :** les métadonnées on-chain (quelques centaines d'octets par transfert : adresses, pointeur, hash) — et d'éventuelles copies de chiffré chez des nœuds qui les ont récupérées, raison précise pour laquelle la confidentialité est déléguée à l'enveloppe hybride PQC plutôt qu'à l'effacement.

C'est aussi ici que les modèles d'expiration du stockage deviennent un critère de sélection de premier rang (section 7) : un backend à expiration native et opposable transforme la sémantique de boîte aux lettres d'une discipline côté client en garantie d'infrastructure.

9. Synthèse du modèle de menace

Adversaire	Défense
Fournisseur de stockage (lit, altère, substitue, retient)	ne voit que du chiffré ; manifestes ancrés au keccak-256 on-chain ; chunks authentifiés AEAD ; la rétention = déni de service seulement
Espion réseau enregistrant tout aujourd'hui	KEM hybride : il faut <i>à la fois</i> une cassure EC-DLP et une cassure ML-KEM pour jamais déchiffrer
Point d'accès RPC malveillant / compromis	annonces vérifiées par recouvrement de signature ; clés du destinataire vérifiées par dérivation d'adresse et engagements on-chain ; un RPC menteur peut cacher des événements (vivacité), pas en forger (sûreté)
Usurpation d'expéditeur	<code>FileSent.from</code> est le signataire de la transaction ; manifestes doublement signés ; la clé classique doit dériver vers l'adresse déclarée
Substitution de manifeste/chunk au même pointeur	hash de contenu on-chain vérifié avant déchiffrement ; tags par chunk
Spam (expéditeurs permissionless)	filtrage à la lecture contre la Blocklist on-chain par destinataire
Adversaire quantique (futur)	confidentialité : le KEM hybride tient. Authenticité du contenu : ML-DSA tient. Plafond connu : la liaison clé→identité reste classique (section 4.3)
Analyse de métadonnées	hors périmètre en v1 — le graphe d'annonces est public par conception ; mixage/relais en feuille de route

10. Intégration agents IA : le protocole comme outils MCP

MCPTTransfer tient son nom de sa surface d'intégration : l'implémentation de référence inclut un **serveur Model Context Protocol** qui expose le protocole complet en dix outils (`whoami`, `resolve`, `whois`, `inbox`, `register_key`, `claim`, `block_sender`, `unblock_sender`, `send_file`, `receive_file`). Tout hôte compatible MCP — assistants de bureau, frameworks d'agents, IDE — peut laisser son modèle exécuter « envoie ce rapport à `alice-ai` » en un seul appel d'outil : résolution du handle, récupération et vérification d'engagement de clé, chiffrement hybride, téléversement et annonce on-chain se déroulent derrière la frontière de l'outil.

L'intégration prend la sécurité de niveau agent au sérieux plutôt que de présumer un hôte bienveillant :

- **Modèle d'autorité explicite** : le serveur détient les clés de l'agent et signe à la demande de l'hôte ; chaque outil qui dépense du gas l'annonce dans sa description, et le *transfert de handle* n'est délibérément pas exposé comme outil (trop d'autorité pour un canal exposé à l'injection de prompt).
- **Confinement du système de fichiers** : une racine de travail opt-in confine ce que `send_file` peut lire et ce que `receive_file` peut écrire, avec une résolution de chemins résistante à la traversée — limitant l'exfiltration ou l'écrasement par un hôte victime d'injection.
- **Signature sûre en concurrence** derrière un verrou processus, et dégradation propre (vérification par signature seule, fenêtres de logs rétrécissantes) quand l'infrastructure de chaîne flanche.

Le fichier d'identité de l'agent au repos supporte le chiffrement par phrase de passe (Argon2id, paramètres RFC 9106, enveloppant de l'AES-256-GCM avec l'en-tête KDF authentifié en données associées), avec une mise à zéro best-effort des clés en mémoire — une garde de clés pragmatique pour des processus d'agents sans surveillance.

11. Implémentation de référence et validation live

Le protocole n'est pas une conception sur papier. Une implémentation de référence complète existe, open-source sur <https://github.com/cabbry/mcpttransfer> (.NET 10 / C# avec BouncyCastle pour ML-KEM/ML-DSA, ~250 lignes de Solidity) :

- **Quatre contrats déployés sur un testnet EVM public** (Polygon Amoy), adresses préconfigurées dans le client.
- **Validation live de bout en bout sur infrastructure banalisée** : un fichier de 18 Mo chiffré en deux chunks, épinglé via un service public d'épingleage IPFS (Pinata), annoncé on-chain, puis reçu **identique à l'octet près** par l'identité destinataire — le client rapportant la réception *corroborée contre le hash de contenu du FileSent on-chain* et l'expéditeur résolu inversement vers son handle revendiqué.
- **Cross-machine, cross-OS, piloté par des agents** : le même échange a été réalisé entre deux machines physiquement distinctes — Windows et macOS — sans aucune connexion directe entre elles, en coordonnant uniquement via la chaîne et IPFS, et piloté de bout en bout sous forme d'appels d'outils Model Context Protocol par un agent IA de chaque côté (le `send_file` d'un agent vers l'adresse de l'autre ; le `receive_file` du destinataire, déchiffré à l'octet près). La promesse du protocole — deux agents autonomes échangeant un fichier sans compte partagé ni serveur central — est ainsi démontrée, pas seulement affirmée.
- **Discipline de test** : ~350 tests unitaires (y compris entrées adverses : manifestes altérés, enveloppes au mauvais destinataire, clés malformées, engagements non concordants, fichiers d'identité forgés), 52 tests de contrats, et une suite d'intégration conditionnelle qui démarre une chaîne locale, déploie, et exécute le pipeline complet enregistrer → revendiquer → envoyer → recevoir en live.
- **Réalisme opérationnel** : le passage en live a révélé et corrigé des problèmes du monde réel qu'un laboratoire ne montre jamais — plafonds de plages de logs et limites de débit des RPC publics, et un bug subtil de vérification de hash null-contre-vide atteignable uniquement quand la corroboration on-chain est indisponible. Les deux sont désormais couverts par des tests de régression.

Le coût de bout en bout d'un transfert sur la chaîne de référence se résume à l'émission d'un événement plus, une fois par agent, l'enregistrement de clés et le nommage — du gas réduit aux métadonnées, indépendant de la taille du fichier ; le coût du plan de données est celui du backend de stockage choisi.

12. Économie et soutenabilité

La couche protocolaire est et restera **libre et ouverte** : pas de token, pas de frais par transfert, pas de contrat de rente. C'est un rejet délibéré des conceptions qui monétisent le point de passage, car le point de passage est exactement ce qu'une économie d'agents permissionless ne peut pas se permettre.

La soutenabilité suit le schéma *protocole ouvert, infrastructure hébergée* (le modèle des fournisseurs SMTP/IMAP, ou de l'épingleage IPFS commercial au-dessus d'un réseau libre) : chacun peut faire tourner toute la pile lui-même depuis l'open source ; des opérateurs (dont les auteurs) peuvent offrir la commodité managée — épingleage fiable avec gestion du cycle de vie, API d'indexeur/passarelle pour que les agents légers évitent de faire tourner RPC + épingleage eux-mêmes, onboarding de flottes pour les déploiements d'agents en entreprise. Aucun de ces services n'est privilégié : ils ne détiennent aucune clé du protocole, et chaque client vérifie tout, quel que soit celui qui le sert.

13. Feuille de route et usage des fonds

Protocole

- Accusés de réception et désépingleage automatique après réception ; sémantique `gc/TTL` standardisée (section 8).
- Adaptateurs de stockage : deals Filecoin à expiration native ; compatible S3 ; réseaux de blobs décentralisés supplémentaires.
- Ports du plan de contrôle au-delà de l'EVM (au moins un écosystème non-EVM) pour étayer l'agnosticisme de chaîne avec du code qui tourne.

- Piste de recherche sur la confidentialité des métadonnées : conceptions de relais/mixage pour le graphe d'annonces.

Assurance

- Audit de sécurité indépendant de l'enveloppe cryptographique et des contrats (l'élément le plus précieux de cette liste).
- Une spécification formelle du protocole (format d'enveloppe, sémantique des registres, règles de vérification) pour que des implémentations indépendantes interopèrent — l'implémentation C# actuelle devenant alors *un* client, pas *le* client.

Écosystème

- SDK au-delà de .NET (TypeScript, Python — les langages où vivent les frameworks d'agents), distribution packagée du serveur MCP.
- Un déploiement d'intérêt général : contrats maintenus sur des chaînes de niveau mainnet plus une passerelle communautaire.

Le financement par grant est recherché spécifiquement pour la piste d'assurance et les ports prouvant l'agnosticisme — ce qui transforme un proof-of-concept fonctionnel en infrastructure neutre, auditable et multi-écosystèmes.

14. Références

- NIST FIPS 203 — *Module-Lattice-Based Key-Encapsulation Mechanism Standard* (ML-KEM), août 2024.
- NIST FIPS 204 — *Module-Lattice-Based Digital Signature Standard* (ML-DSA), août 2024.
- NIST SP 800-38D — *Galois/Counter Mode (GCM) and GMAC*.
- NIST SP 800-227 (draft) — *Recommendations for Key-Encapsulation Mechanisms*.
- RFC 5869 — *HKDF: HMAC-based Extract-and-Expand Key Derivation Function*.
- RFC 9106 — *Argon2 Memory-Hard Function for Password Hashing*.
- Draft IETF — *X-Wing: general-purpose hybrid post-quantum KEM* (modèle structurel de la construction hybride).
- RFC 6979 — *Deterministic Usage of DSA and ECDSA*.
- Model Context Protocol — <https://modelcontextprotocol.io>.
- BSI TR-02102-1 — *Cryptographic Mechanisms* (recommandations PQC hybrides).

Contact : Jean-Romain Bouquet — cabbry@icloud.com. Source, suites de tests et déploiement testnet live : <https://github.com/cabbry/mcptransfer>.